

0.112. La fonction `volumeSon()` — le volume général

`#include <volumeSon>` : `volumeSon(0 à 7)` règle le volume de toute la console.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `volumeSon(n)` règle le **volume général**, de 0 (muet) à 7 (le plus fort), pour toutes les voix ensemble. En le baissant pas à pas, une musique s'éteint en douceur.

La ligne à écrire : `#include <volumeSon>`.

Son argument : de 0 à 7.

Ce qu'elle coûte : une vingtaine d'octets.

Essaie : baisse-le de 7 à 0, d'un cran toutes les 250 ms : un fondu.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **538 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      61 o ← la console, à l'allumage – la mise en route de la console
main()         83 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   49 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles        128 o ← CopierTuiles – 8 lettres sur 43
Notes         120 o ← note() – la table des notes de musique
"DOUCEMENT"    9 o ← texte() – un texte gravé
```

Lettres gravées : CDEMNOTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert

Fonctions du langage appelées :

```
note()      x 1   55 o sur place, dans main() + Notes (120 o)
texte()     x 1   11 o sur place, dans main() + EcrireTexte (10 o)
volumeSon() x 1   10 o sur place, dans main()
image()     x 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

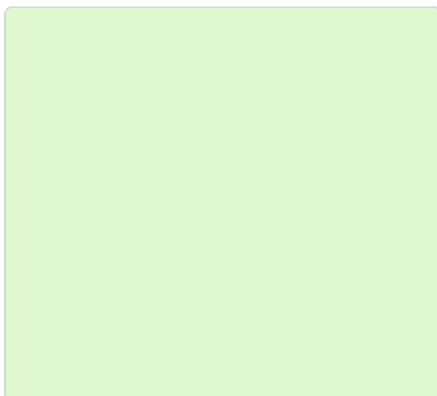
```
// ---- #include <volumeSon> : le volume général ----  
  
#include <note>  
#include <volumeSon> // volumeSon() : le volume de toute la console  
#include <texte>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(5, 8, "DOUCEMENT");  
    volumeSon(3);           // à peu près à moitié : 3 sur 7  
    note(1, MI4, 0, 15);    // une note forte... mais tout est à moitié  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un MI qui sonne doucement.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le volume général vaut 3, à gauche comme à droite (33)
- ✓ la note joue

Ensuite : **0.113. #include <decaler> — a << b et a >> b** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++